

---

## PROJECT REPORT

# Production-Grade Serverless URL Shortener & Analytics Platform

A serverless, event-driven link-shortening service on AWS — API Gateway, Lambda, DynamoDB, SQS, and an S3-backed analytics data lake, deployed with Terraform and validated by a 41-test automated suite.

---

**Shyam Kumar D**

Aspiring Cloud Architect | AWS Cloud, Serverless & Infrastructure Engineering

[linkedin.com/in/shyam-kumar-d](https://www.linkedin.com/in/shyam-kumar-d)

[github.com/ShyamD2](https://github.com/ShyamD2)

Live Repository: [github.com/ShyamD2/aws-cloud-serverless-url-shortener](https://github.com/ShyamD2/aws-cloud-serverless-url-shortener)

September 2026

## 1. Executive Summary

This project is a production-style, fully serverless URL shortener and click-analytics platform built on AWS. Every request path — link creation, redirection, and click tracking — runs on managed, auto-scaling services with no servers to patch or provision: Amazon API Gateway for HTTP routing, AWS Lambda (Python 3.13) for compute, DynamoDB for low-latency link lookups, and an SQS-decoupled pipeline that streams click telemetry into a partitioned S3 analytics data lake.

The goal was to go beyond a toy shortener and demonstrate the engineering practices a real production system needs: asynchronous processing so redirect latency never blocks on analytics, a dead-letter queue for poison-pill resilience, API throttling to protect the backend, a live CloudWatch operations dashboard, least-privilege IAM roles scoped per function, Infrastructure-as-Code with Terraform (remote state + state locking), and an automated pytest suite covering validation, handlers, short-code generation, and the analytics pipeline.

This report walks through the deployed system component by component, in the order a reviewer would naturally trace a request: from the live frontend, through the API and compute layer, into the data and analytics stores, and finally the IaC and IAM foundations underneath.

**Live Repository:** [github.com/ShyamD2/aws-cloud-serverless-url-shortener](https://github.com/ShyamD2/aws-cloud-serverless-url-shortener) — full source, Terraform configuration, and the pytest suite referenced throughout this report.

## 2. Architecture Overview

The system follows an event-driven, fully managed serverless pattern deployed in the Asia Pacific (Mumbai) region. A user creates a short link through the frontend, which calls API Gateway; the **dev-create-url** Lambda validates the payload and writes a record to DynamoDB. When the short link is visited, the **dev-redirect** Lambda looks up the record, issues a 302 redirect, and publishes a click event to an SQS queue instead of writing analytics inline — keeping redirects fast. The **dev-analytics-processor** Lambda consumes batches from SQS and persists them as partitioned JSONL objects in S3 for downstream analysis.

| Component       | AWS Service              | Purpose                                                                                                       |
|-----------------|--------------------------|---------------------------------------------------------------------------------------------------------------|
| Frontend        | S3 static hosting        | Serves the link-creation UI and live telemetry dashboard (app.js, index.html, style.css)                      |
| API Layer       | API Gateway (HTTP API)   | Exposes create / redirect / delete routes; enforces throttling (burst 50, rate 100/route)                     |
| Compute         | AWS Lambda (Python 3.13) | dev-create-url, dev-redirect, dev-delete-url, dev-analytics-processor                                         |
| Data Store      | DynamoDB                 | dev-urls table — short_code, original_url, click_count, TTL/expiry, status                                    |
| Async Telemetry | SQS + DLQ                | dev-click-events queue decouples redirect latency from analytics processing; DLQ catches poison-pill messages |
| Analytics Lake  | S3 (partitioned JSONL)   | Click events written under clicks/year=/month=/day=/hour= for downstream querying                             |
| Observability   | CloudWatch Dashboard     | Tracks API traffic/errors, SQS queue depth, Lambda latency (p95), DynamoDB capacity                           |
| IAM             | Least-privilege roles    | One dedicated execution role per Lambda function                                                              |

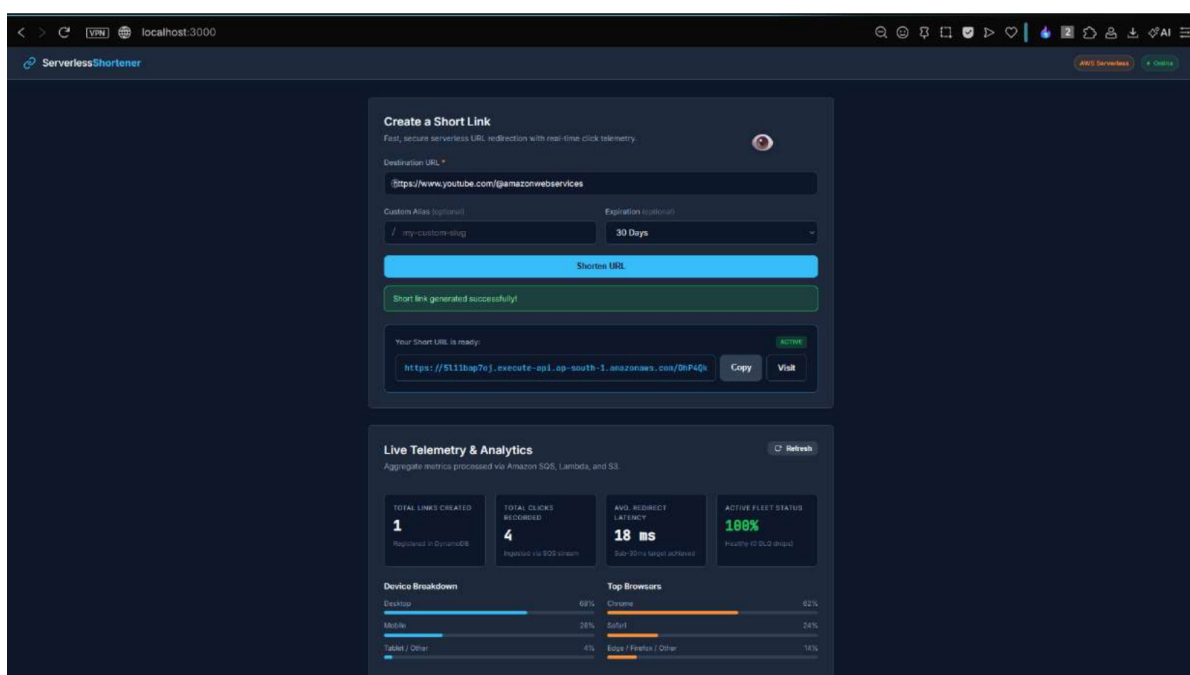
|     |           |                                                                             |
|-----|-----------|-----------------------------------------------------------------------------|
| laC | Terraform | Remote state in S3 with DynamoDB state locking for safe, repeatable deploys |
|-----|-----------|-----------------------------------------------------------------------------|

### 3. Step-by-Step Build with Screenshots

The screenshots below are presented in the order a request actually flows through the system: frontend → API/compute → async analytics pipeline → data stores → infrastructure-as-code and IAM foundations underneath. Sensitive account-level identifiers visible in a couple of screenshots are left as captured from the live AWS account used for this build.

#### 3.1 — Frontend — Short Link Creation & Live Telemetry

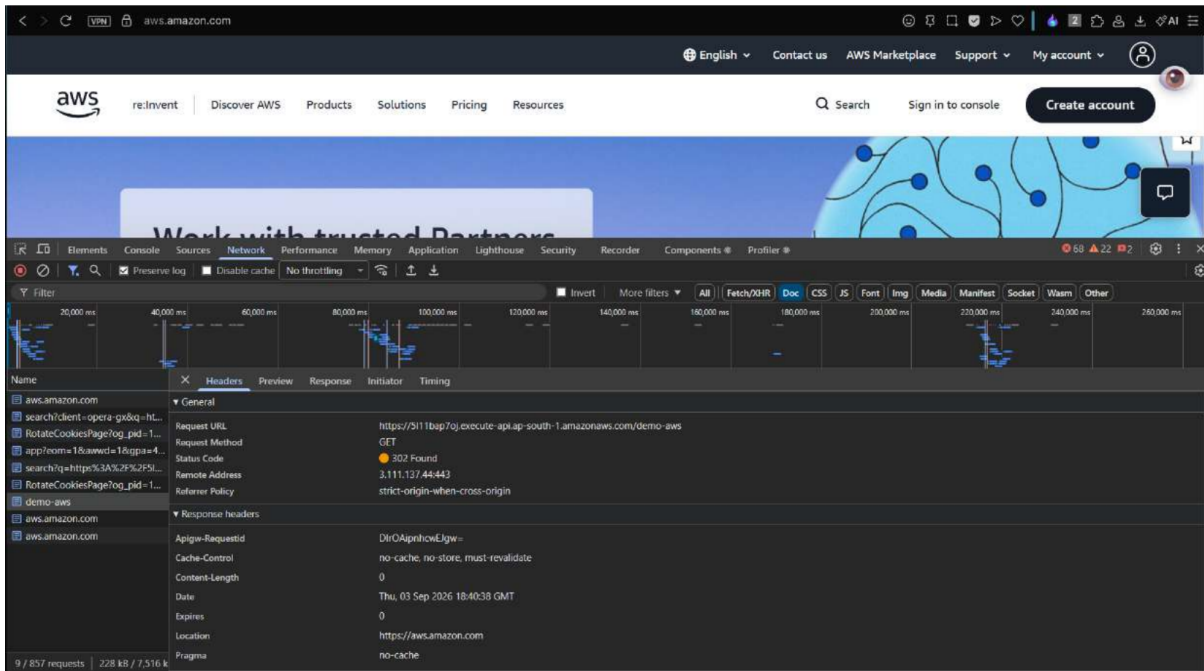
The React/JS frontend, hosted as a static site on S3, lets a user submit a destination URL with an optional custom alias and expiration window. On submission it calls the API Gateway endpoint and displays the generated short URL. The lower panel — Live Telemetry & Analytics — polls the backend for aggregate metrics (links created, total clicks, average redirect latency, active fleet status) along with a device and browser breakdown, all sourced from the SQS → Lambda → S3 analytics pipeline described in Section 2.



Frontend dashboard showing a successfully generated short link and live click telemetry

### 3.2 — End-to-End Redirect Verification (Browser DevTools)

This is the proof that the shortener works end-to-end in a live browser: the Network tab shows a GET request to the API Gateway invoke URL returning **302 Found**, with a **Location** response header pointing to the original destination (aws.amazon.com). This confirms the dev-redirect Lambda correctly resolves the short code against DynamoDB and issues a standards-compliant HTTP redirect.

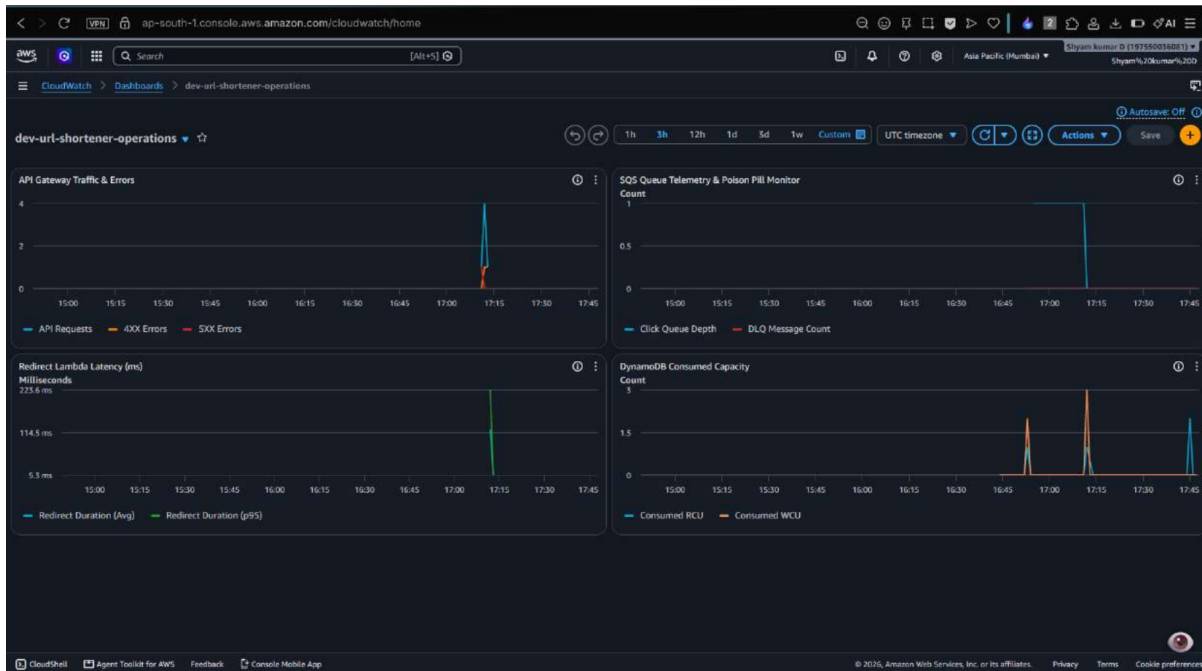


DevTools Network tab confirming a 302 redirect served by the API Gateway + Lambda redirect path



### 3.3 — CloudWatch Operations Dashboard

A custom CloudWatch dashboard (**dev-url-shortener-operations**) gives a single-pane view of system health: API Gateway traffic and 4XX/5XX errors, SQS click-queue depth alongside the dead-letter queue's poison-pill count, redirect Lambda latency (average and p95), and DynamoDB consumed read/write capacity. This is the operational visibility layer that lets an engineer confirm the system is healthy without digging through raw logs.



CloudWatch dashboard — API traffic/errors, SQS telemetry, Lambda latency, and DynamoDB capacity

### 3.4 — Automated Test Suite — 41/41 Passed

The backend is covered by a pytest suite of 41 tests spanning analytics (user-agent parsing, referrer sanitization, batch publishing to S3), request handlers (create/redirect/delete success and error paths), short-code generation and custom-alias validation, the URL service layer (duplicate-alias handling, expiration, soft-delete), and payload validation (target URL checks, missing-scheme rejection, loop prevention, TTL bounds). All 41 tests pass in under 4 seconds, giving confidence to iterate on the Lambda code with a fast feedback loop.

```
C:\Windows\System32\cmd.exe X + v
===== test session starts =====
platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.6.0 -- E:\downloads\Production-Grade Serverless URL Shortener & Analytics Platform\.venv\Scripts\python.exe
cachedir: pytest_cache
rootdir: E:\downloads\Production-Grade Serverless URL Shortener & Analytics Platform
configfile: pytest.ini
testpaths: application/tests
plugins: mock-3.15.1
collected 41 items

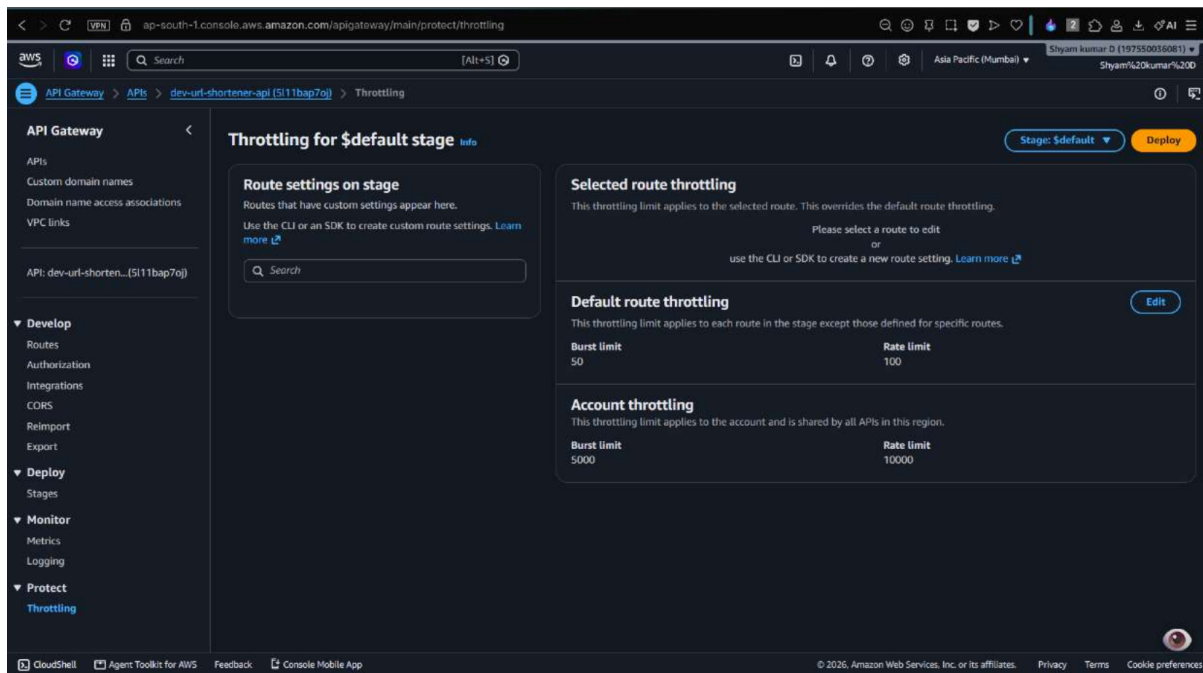
application/tests/test_analytics.py::test_parse_user_agent_desktop_chrome PASSED [ 24%]
application/tests/test_analytics.py::test_parse_user_agent_mobile_iphone PASSED [ 41%]
application/tests/test_analytics.py::test_parse_user_agent_bot PASSED [ 58%]
application/tests/test_analytics.py::test_parse_user_agent_empty PASSED [ 76%]
application/tests/test_analytics.py::test_sanitize_referrer PASSED [ 94%]
application/tests/test_analytics.py::test_analytics_service_publish PASSED [ 12%]
application/tests/test_analytics.py::test_analytics_handler_processes_batch_to_s3 PASSED [ 24%]
application/tests/test_handlers.py::test_handler_create_url_success PASSED [ 36%]
application/tests/test_handlers.py::test_handler_create_url_invalid_body PASSED [ 58%]
application/tests/test_handlers.py::test_handler_create_url_validation_error PASSED [ 70%]
application/tests/test_handlers.py::test_handler_create_url_alias_conflict PASSED [ 82%]
application/tests/test_handlers.py::test_handler_redirect_success PASSED [ 94%]
application/tests/test_handlers.py::test_handler_redirect_not_found PASSED [ 100%]
application/tests/test_handlers.py::test_handler_redirect_disabled PASSED [ 100%]
application/tests/test_handlers.py::test_handler_delete_url_success PASSED [ 100%]
application/tests/test_handlers.py::test_handler_delete_url_not_found PASSED [ 100%]
application/tests/test_short_code.py::test_generate_short_code_default_length PASSED [ 100%]
application/tests/test_short_code.py::test_generate_short_code_custom_length PASSED [ 100%]
application/tests/test_short_code.py::test_generate_short_code_invalid_length PASSED [ 100%]
application/tests/test_short_code.py::test_generate_short_code_uniqueness PASSED [ 100%]
application/tests/test_short_code.py::test_is_valid_custom_alias_valid PASSED [ 100%]
application/tests/test_short_code.py::test_is_valid_custom_alias_too_short PASSED [ 100%]
application/tests/test_short_code.py::test_is_valid_custom_alias_too_long PASSED [ 100%]
application/tests/test_short_code.py::test_is_valid_custom_alias_reserved_keywords PASSED [ 100%]
application/tests/test_url_service.py::test_create_url_generated_code PASSED [ 100%]
application/tests/test_url_service.py::test_create_url_custom_alias PASSED [ 100%]
application/tests/test_url_service.py::test_create_url_duplicate_alias_raises PASSED [ 100%]
application/tests/test_url_service.py::test_create_url_with_expiration PASSED [ 100%]
application/tests/test_url_service.py::test_delete_url_not_found PASSED [ 100%]
application/tests/test_url_service.py::test_delete_url_marked_disabled PASSED [ 100%]
application/tests/test_validation.py::test_validate_target_url_valid PASSED [ 100%]
application/tests/test_validation.py::test_validate_target_url_invalid_schemes PASSED [ 100%]
application/tests/test_validation.py::test_validate_target_url_missing_domain PASSED [ 100%]
application/tests/test_validation.py::test_validate_target_url_too_long PASSED [ 100%]
application/tests/test_validation.py::test_validate_target_url_loop_prevention PASSED [ 100%]
application/tests/test_validation.py::test_validate_ttl_days PASSED [ 100%]
application/tests/test_validation.py::test_validate_create_payload_success PASSED [ 100%]
application/tests/test_validation.py::test_validate_create_payload_invalid_body PASSED [ 100%]

===== 41 passed in 3.78s =====
```

pytest run — 41 passed in 3.78s, covering handlers, validation, short-code logic, and analytics

### 3.5 — API Gateway Throttling & Rate Limiting

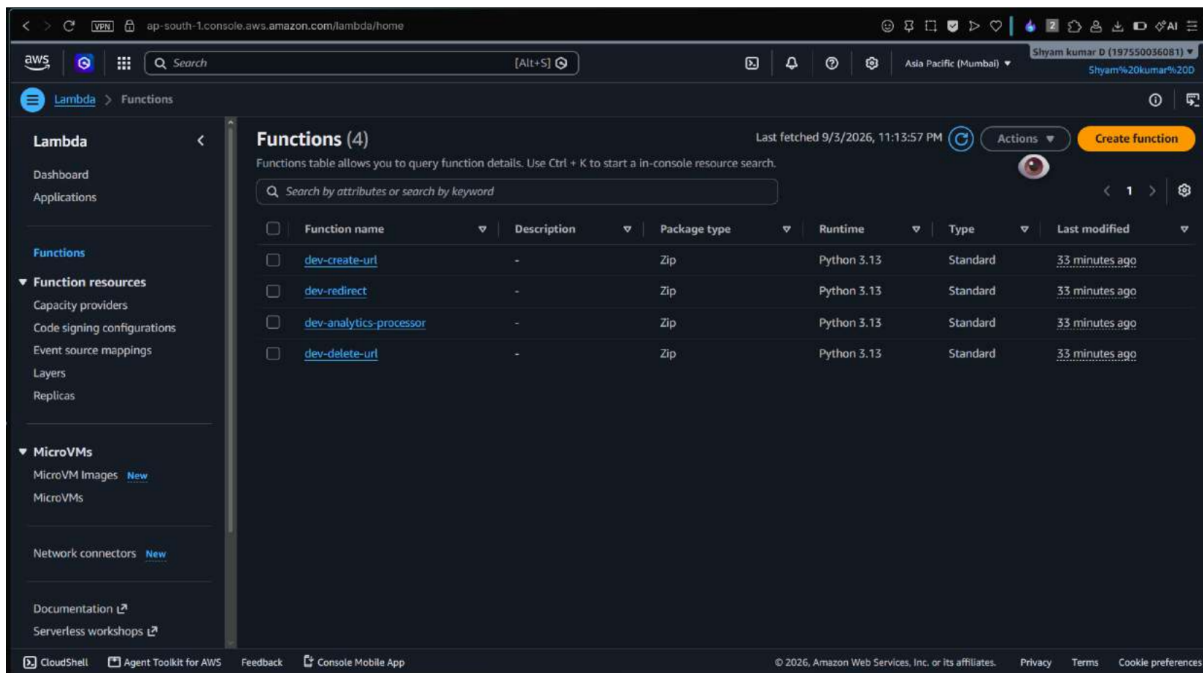
To protect the backend from abuse or traffic spikes, the API Gateway **\$default** stage is configured with default route throttling of a 50 request burst limit and a 100 request/second steady-state rate limit, sitting under the account-level ceiling of a 5,000 burst / 10,000 requests-per-second limit. This is a low-cost, managed alternative to standing up a dedicated rate-limiting layer.



API Gateway throttling configuration for the \$default stage

### 3.6 — Lambda Functions Deployed

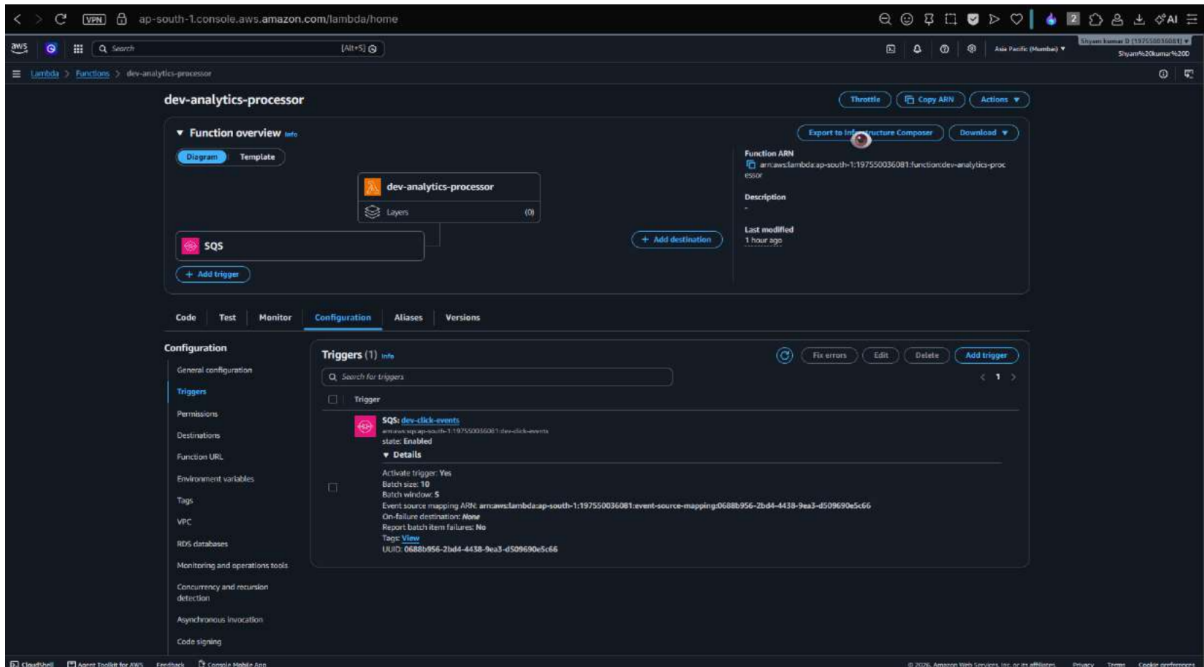
The compute layer is split into four single-purpose Lambda functions, all running Python 3.13: **dev-create-url** (validates and persists new short links), **dev-redirect** (resolves a short code and issues the 302), **dev-analytics-processor** (consumes click events from SQS), and **dev-delete-url** (soft-deletes/disables a link). Keeping each function narrowly scoped keeps cold-start size small and makes IAM permissions easy to reason about per function.



*Lambda console — the four functions that make up the create / redirect / analytics / delete flow*

### 3.7 — Analytics Processor — SQS Trigger Configuration

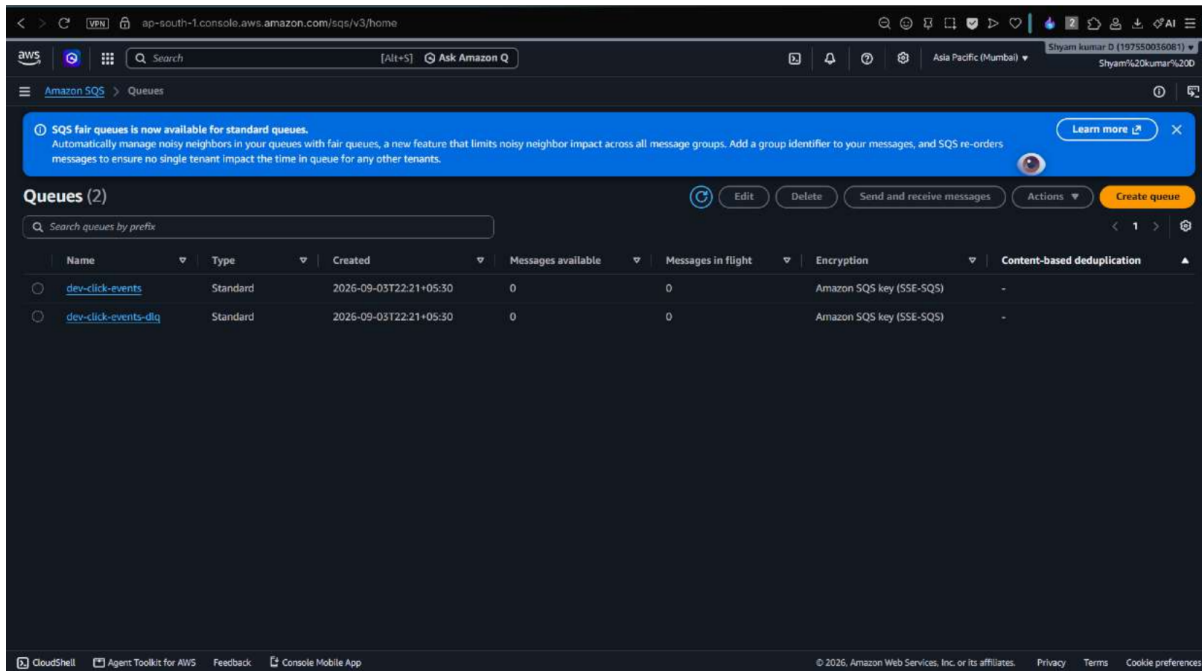
The **dev-analytics-processor** function is wired directly to the **dev-click-events** SQS queue as an event source, with a batch size of 10 and a 5-second batch window. This lets the function process click events in small, efficient batches rather than invoking once per click, reducing invocation count and cost while keeping processing latency low.



Lambda trigger configuration — SQS event source mapping for the analytics processor

### 3.8 — SQS Queues — Click Events & Dead-Letter Queue

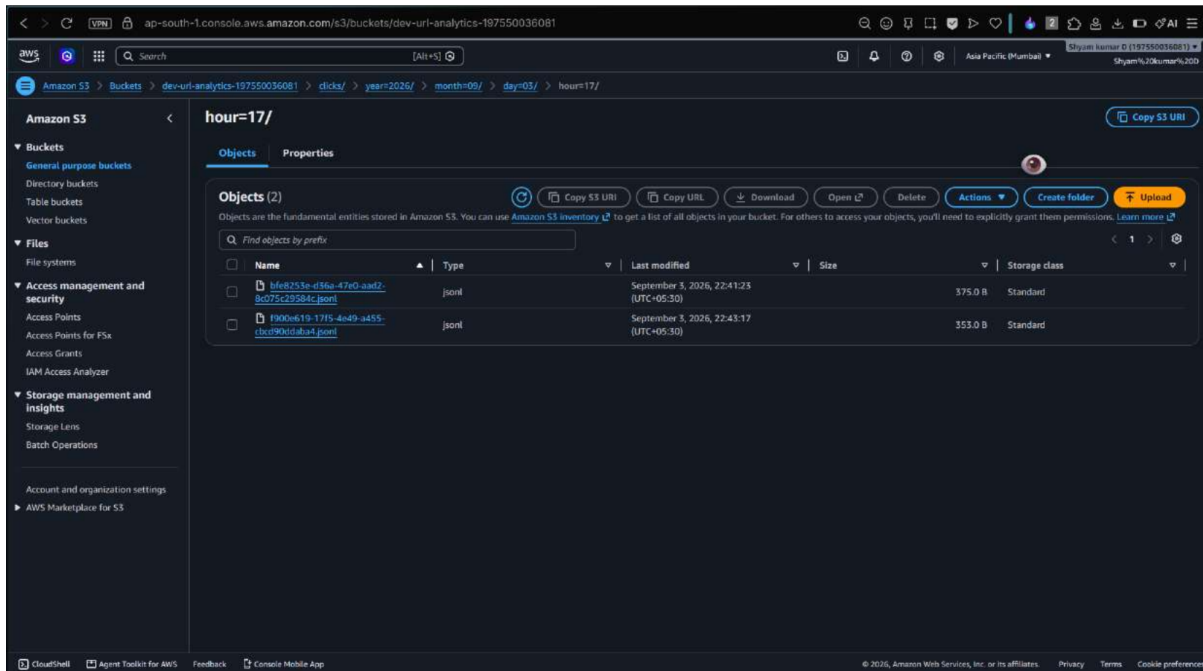
Two standard SQS queues back the async pipeline: **dev-click-events**, which receives a message for every redirect, and **dev-click-events-dlq**, its dead-letter queue. If a message fails processing repeatedly, it is automatically routed to the DLQ instead of blocking or silently dropping data — the same poison-pill count tracked on the CloudWatch dashboard in Section 3.3. Both queues use SQS-managed (SSE-SQS) encryption at rest.



SQS console showing the click-events queue and its dead-letter queue

### 3.9 — S3 Analytics Data Lake — Partitioned Click Events

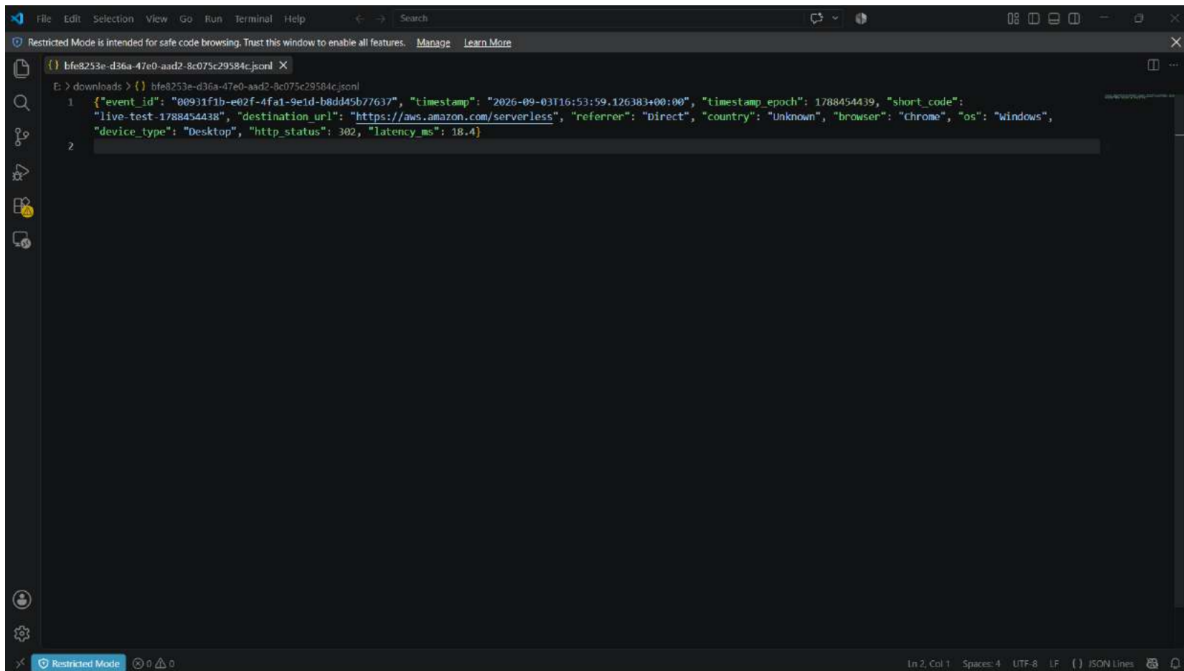
Processed click events are written to the **dev-url-analytics** S3 bucket using an hour-level Hive-style partition scheme — clicks/year=2026/month=09/day=03/hour=17/ — with one JSONL object per analytics batch. This partition layout is what makes the data efficiently queryable later with tools like Athena, without scanning the entire dataset for a single day's traffic.



*S3 analytics bucket — click events partitioned by year/month/day/hour*

### 3.10 — Raw Click Event Record

A single JSONL record from the analytics bucket shows the shape of the data captured per click: an event ID, ISO and epoch timestamps, the short code, destination URL, referrer, parsed country/browser/OS/device type, the HTTP status returned, and end-to-end redirect latency in milliseconds. This structured, self-describing schema is what powers the frontend's device and browser breakdown shown in Section 3.1.

A screenshot of the Visual Studio Code editor interface. The top menu bar includes File, Edit, Selection, View, Go, Run, Terminal, and Help. Below the menu, a status bar indicates 'Restricted Mode is intended for safe code browsing. Trust this window to enable all features.' with links to 'Manage' and 'Learn More'. The main editor area shows a file named 'file0253e-d36a-47e0-aad2-8c075c29384c.jsonl'. The content of the file is a single JSONL record: 

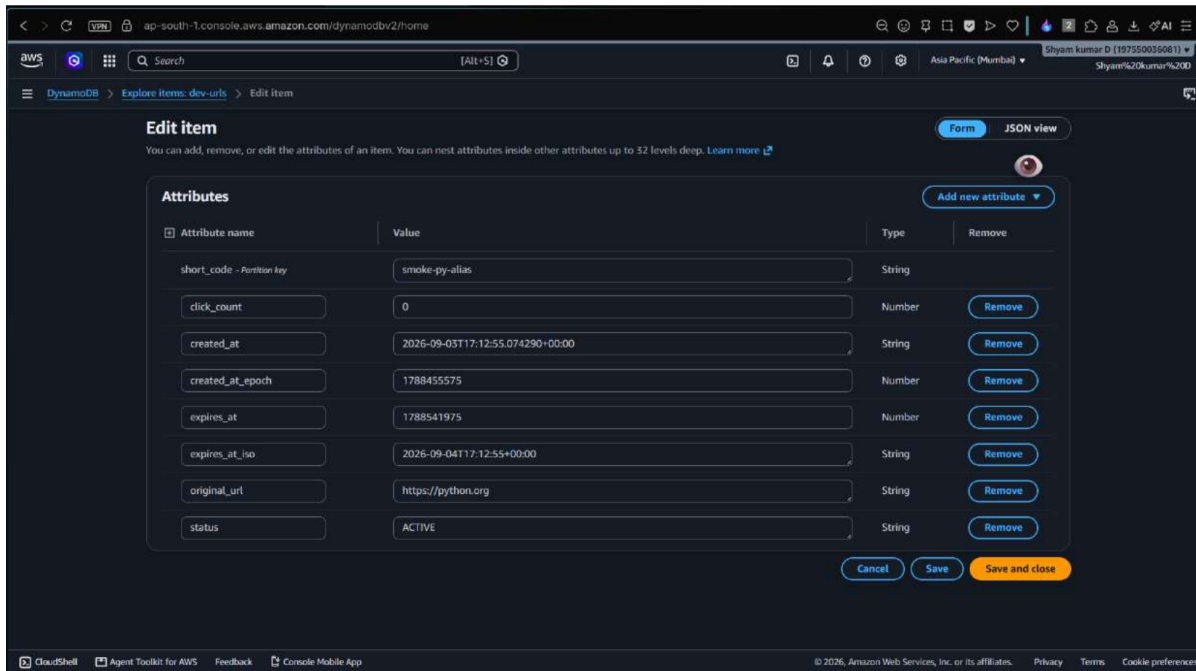
```
1 {"event_id": "80931f1b-e02f-4fa8-9e1d-b8d49b77637", "timestamp": "2026-09-03T16:53:59.126383+00:00", "timestamp_epoch": 1788454439, "short_code":  
"live-test-1788454438", "destination_url": "https://aws.amazon.com/serverless", "referrer": "direct", "country": "Unknown", "browser": "chrome", "os": "windows",  
"device_type": "Desktop", "http_status": 302, "latency_ms": 16.4}
```

*A single click-event record from the S3 analytics data lake, viewed in VS Code*



### 3.11 — DynamoDB — URL Record Structure

The **dev-urls** DynamoDB table uses **short\_code** as its partition key. Each item stores **click\_count**, **created\_at** / **created\_at\_epoch**, **expires\_at** / **expires\_at\_iso**, the **original\_url**, and a **status** flag (ACTIVE/disabled). Storing both epoch and ISO timestamps keeps the table efficient for TTL-style expiry logic while remaining human-readable for debugging in the console.

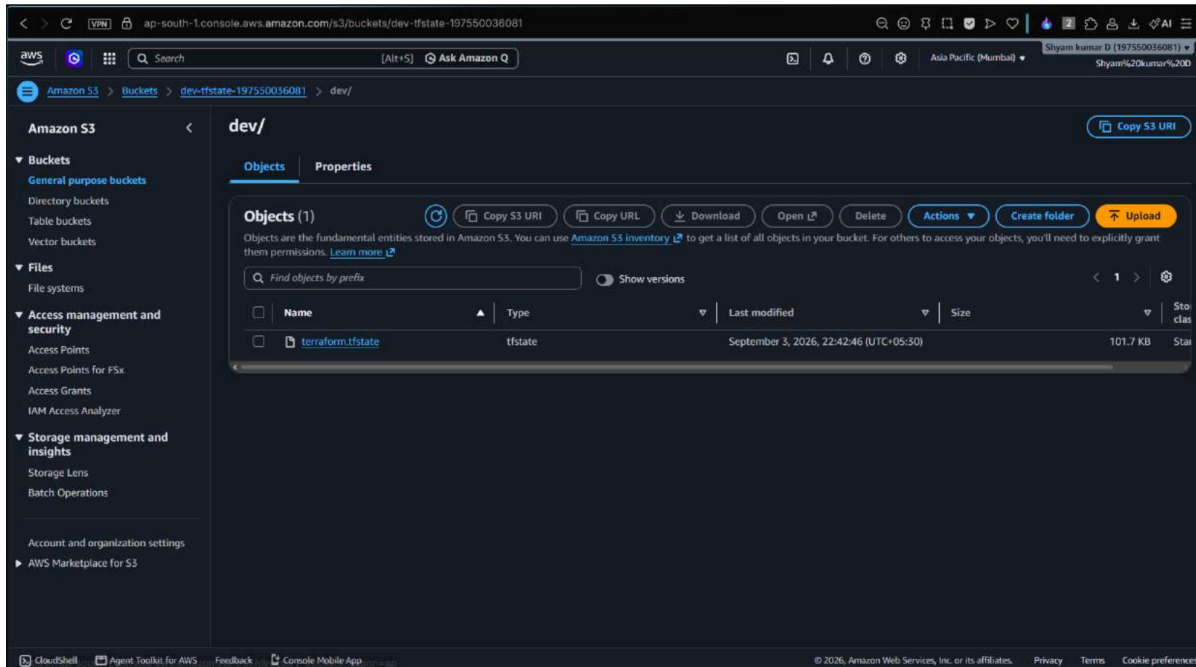


| Attribute name             | Value                            | Type   | Remove |
|----------------------------|----------------------------------|--------|--------|
| short_code - Partition key | smoke-py-alias                   | String |        |
| click_count                | 0                                | Number | Remove |
| created_at                 | 2026-09-03T17:12:55.074290+00:00 | String | Remove |
| created_at_epoch           | 1788455575                       | Number | Remove |
| expires_at                 | 1788541975                       | Number | Remove |
| expires_at_iso             | 2026-09-04T17:12:55+00:00        | String | Remove |
| original_url               | https://python.org               | String | Remove |
| status                     | ACTIVE                           | String | Remove |

*DynamoDB console — editing a dev-urls item and its full attribute set*

### 3.12 — Terraform Remote State (S3 Backend)

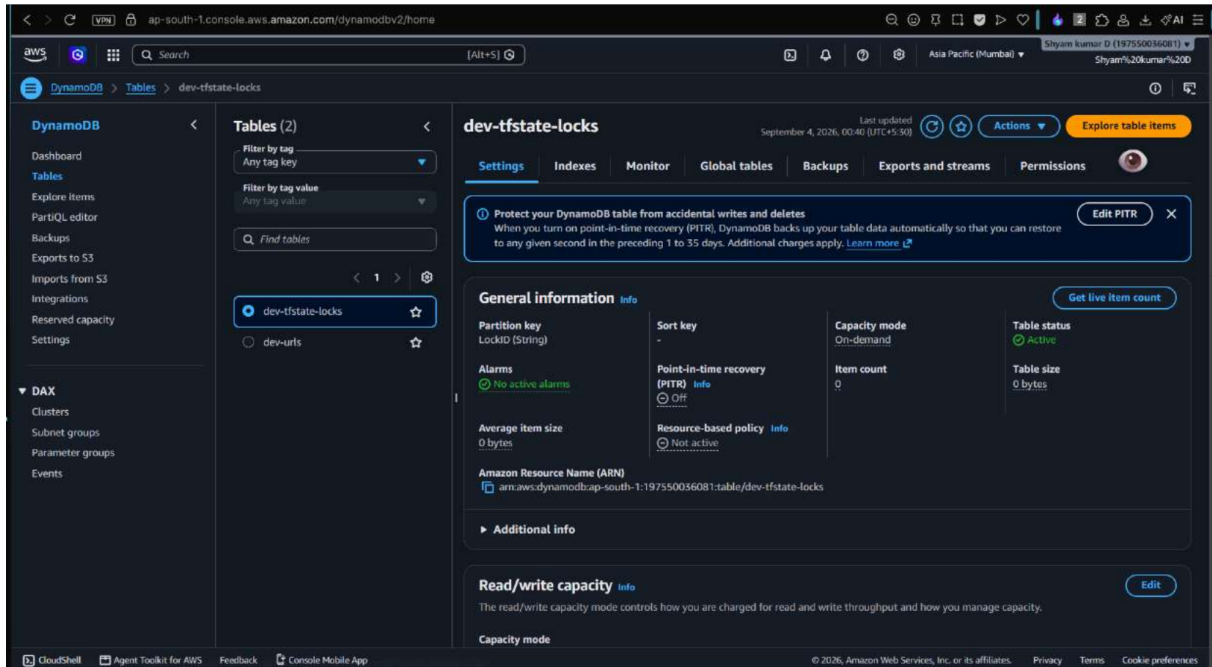
All infrastructure is provisioned with Terraform, and the state file is stored remotely in a dedicated **dev-tfstate** S3 bucket rather than kept locally. This is a standard production practice: it lets state be shared safely across machines and CI runs and keeps a durable, versioned record of everything Terraform manages.



*S3 bucket holding the Terraform remote state file for the environment*

### 3.13 — Terraform State Locking (DynamoDB)

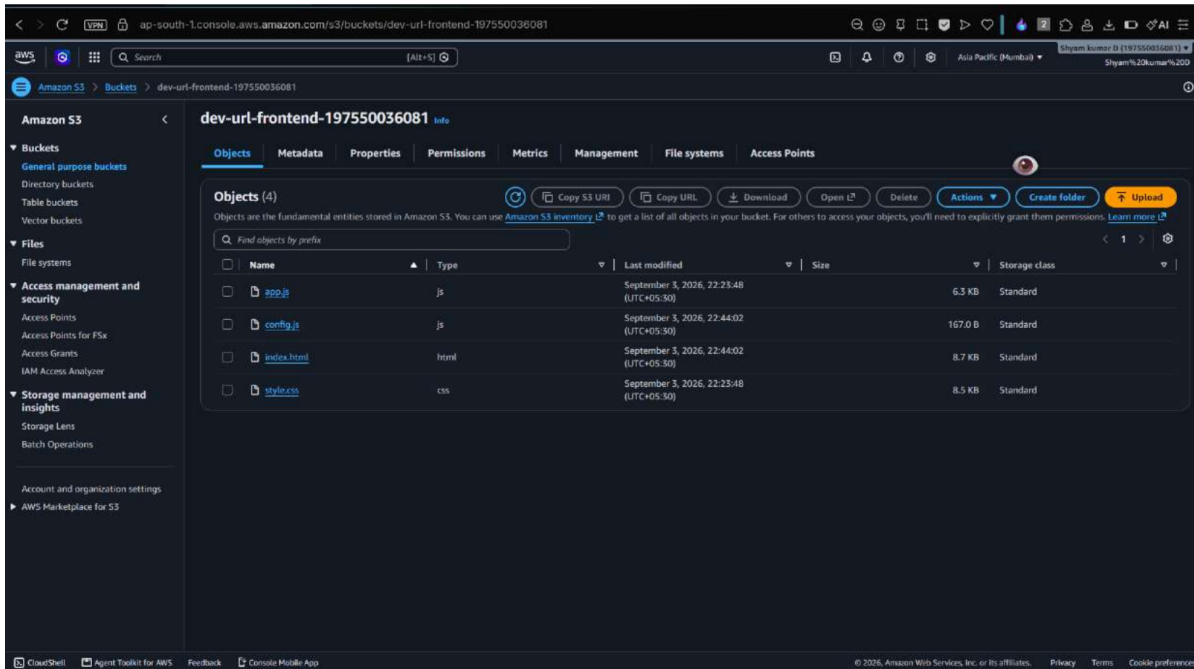
A dedicated **dev-tfstate-locks** DynamoDB table (on-demand capacity, LockID partition key) provides state locking for Terraform operations, preventing two concurrent terraform apply runs from corrupting shared state — the standard S3 + DynamoDB Terraform backend pattern.



*DynamoDB table used to lock Terraform state during apply/plan operations*

### 3.14 — Frontend Hosting Bucket

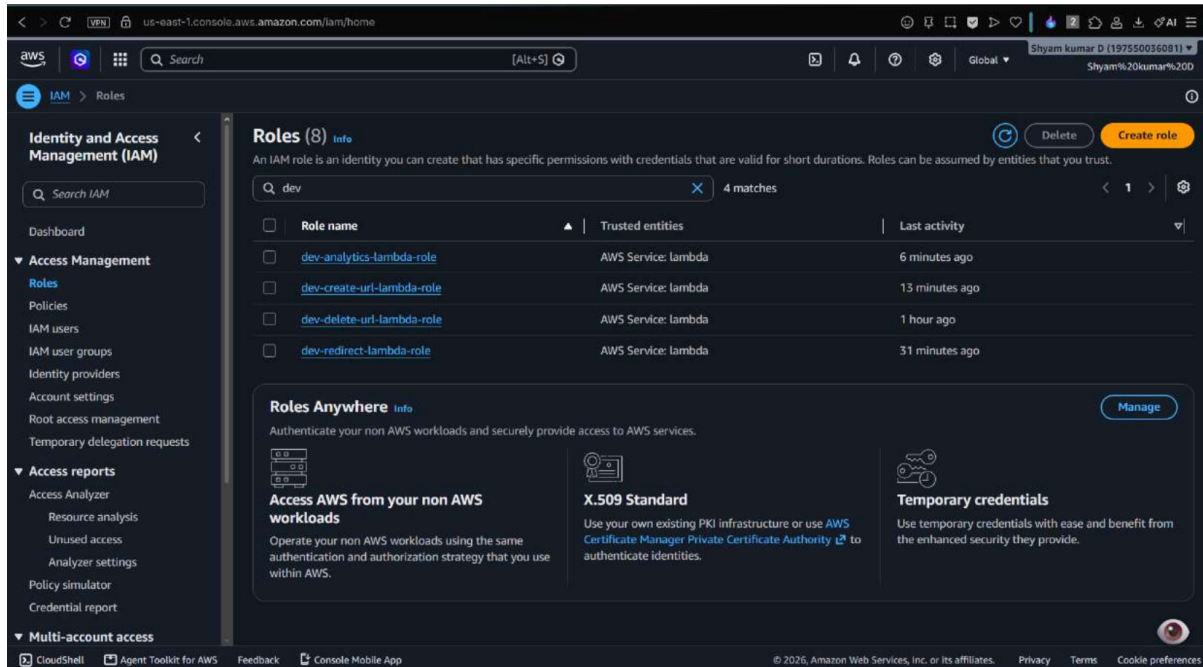
The static frontend assets — **index.html**, **app.js**, **config.js**, and **style.css** — are hosted directly from the **dev-url-frontend** S3 bucket shown in Section 3.1's dashboard. Separating frontend and backend infrastructure into their own buckets keeps deployment and permissions cleanly scoped.



*S3 bucket serving the static frontend for the shortener UI*

### 3.15 — IAM — Least-Privilege Execution Roles

Every Lambda function runs under its own dedicated IAM role — **dev-create-url-lambda-role**, **dev-redirect-lambda-role**, **dev-analytics-lambda-role**, and **dev-delete-url-lambda-role** — rather than a single shared execution role. This enforces least privilege: the redirect function, for example, has no reason to hold S3 write permissions, so it isn't granted any.



*IAM console — one scoped execution role per Lambda function*

## 4. Key Design Considerations

**Decoupling redirects from analytics.** The redirect Lambda never writes analytics data synchronously — it publishes a lightweight message to SQS and returns the 302 immediately, so a slow or failing analytics path can never add latency to a user-facing redirect.

**Resilience via dead-letter queue.** The dev-click-events-dlq isolates messages that repeatedly fail processing, so a malformed event can't block the queue or silently disappear — it stays visible and inspectable, and the CloudWatch dashboard tracks its count directly.

**Cost and blast-radius control.** API Gateway throttling caps abusive traffic before it reaches Lambda/DynamoDB, and per-function IAM roles limit what a compromised or buggy function could actually do.

**Repeatable infrastructure.** Terraform with an S3 + DynamoDB remote backend means the entire environment can be destroyed and rebuilt from code, with state safely shared and locked against concurrent changes.

## 5. Conclusion and Key Takeaways

Building this project end-to-end — from the API and data layer through the async analytics pipeline, observability, IAM, and Infrastructure-as-Code — gave me hands-on practice with the same architectural questions a real production serverless system has to answer: how to keep the hot path fast, how to make failure visible instead of silent, how to scope permissions tightly, and how to make infrastructure changes safe and repeatable.

As I continue building toward a cloud architecture role, this project reflects the direction I want to keep growing in: designing systems that are not just functional, but observable, resilient, and operable at production standards. Next steps I plan to explore include adding Athena/QuickSight on top of the S3 analytics lake for ad-hoc querying and dashboards, introducing CI/CD for the Terraform and Lambda deployment pipeline, and layering in custom domain support with CloudFront and ACM.